PSYC559 Project 1, Part 2: NHIS 2025

Neha Panda

2026-10-17

Contents

1. Define the Research Question 1
2. Recode Your Variables 1
3. Handle Missing Data 8
4. Build the Modeling Pipeline 11
5. Fit and Compare Models 12

```
library(naniar)   # finding and recoding missing values
library(dplyr)    # data manipulation and the pipe
library(ggplot2)  # plots
library(rsample)  # splitting data into training and test sets
library(recipes)  # imputation, dummy coding, and scaling
library(caret)    # train(), cross validation, and performance metrics
library(glmnet)   # ridge, lasso, and elastic net
```

1. Define the Research Question

State what you will predict, and from what. Identify the problem the model is meant to solve. Explain why the problem matters, how a prediction would help, and who would use it. State whether your target is continuous or dichotomous, and thus whether you use linear or logistic regression.

This project will be predicting moderate-to-severe PHQ-8 depression from routine demographic/health variables. The question of whether or not a person will have moderate-to-severe PHQ-8 depression ("yes"/"no") is a dichotomous target, so we will be using logistic regression.

2. Recode Your Variables

Run `str()` and compare each variable to its codebook.

Remove columns that should not be predictors, such as ID numbers, names, and dates of data collection.

Remove any variable that would not be known at the time you make the prediction, or that is another version of the target to avoid data leakage about the outcome into the model.

```
csv_file <- "adult25.csv"
zip_file <- "adult25csv.zip"
zip_url  <- "https://ftp.cdc.gov/pub/Health_Statistics/NCHS/Datasets/NHIS/2025/adult25csv.zip"

if (!file.exists(csv_file)) {
  download.file(zip_url, zip_file, mode = "wb")
  unzip(zip_file)
}
```

```r
dat <- read.csv(csv_file)
dim(dat)
```

```
## [1] 24215   577
```

```r
# define your variable set FIRST
vars <- c("PHQCAT_A", "AGEP_A", "SEX_A", "EDUCP_A", "RATCAT_A",
          "MARSTAT_A", "BMICAT_A", "SMKCIGST_A", "DISAB3_A",
          "PHSTAT_A", "REGION")

str(dat[vars])
```

```
## 'data.frame':    24215 obs. of  11 variables:
##  $ PHQCAT_A  : int  3 1 2 1 1 1 2 2 1 1 ...
##  $ AGEP_A    : int  71 67 19 71 55 52 80 23 45 74 ...
##  $ SEX_A     : int  2 1 1 1 2 2 2 2 2 2 ...
##  $ EDUCP_A   : int  1 3 2 4 8 4 1 8 8 4 ...
##  $ RATCAT_A  : int  6 12 1 14 13 8 14 14 14 7 ...
##  $ MARSTAT_A : int  1 5 7 1 7 1 1 7 1 4 ...
##  $ BMICAT_A  : int  3 2 3 3 4 9 2 3 4 2 ...
##  $ SMKCIGST_A: int  3 3 4 4 4 4 1 4 4 1 ...
##  $ DISAB3_A  : int  1 2 2 2 2 2 2 2 2 2 ...
##  $ PHSTAT_A  : int  3 4 3 4 2 1 4 1 2 4 ...
##  $ REGION    : int  3 3 3 3 3 3 3 3 3 3 ...
```

```r
lapply(dat[vars], function(x) table(x, useNA = "always"))
```

```
## $PHQCAT_A
## x
##     1     2     3     4     8  <NA>
## 18604  3352  1083   580   596     0
##
## $AGEP_A
## x
##   18   19   20   21   22   23   24   25   26   27   28   29   30   31   32   33
##  182  166  189  168  185  247  241  265  296  292  300  338  350  370  355  393
##   34   35   36   37   38   39   40   41   42   43   44   45   46   47   48   49
##  401  385  384  385  373  369  366  379  394  367  345  338  323  323  310  340
##   50   51   52   53   54   55   56   57   58   59   60   61   62   63   64   65
##  308  294  317  353  349  341  334  342  378  369  398  435  458  434  458  484
##   66   67   68   69   70   71   72   73   74   75   76   77   78   79   80   81
##  475  462  509  489  493  430  448  386  390  367  369  387  399  266  269  265
##   82   83   84   85   97   99 <NA>
##  227  221  169  963   56    4    0
##
## $SEX_A
## x
##     1     2     7     9  <NA>
## 11215 12975     2    23     0
##
## $EDUCP_A
## x
##    1    2    3    4    5    6    7    8    9   10   97   99 <NA>
## 1428  377  572 5523 3391  955 2309 5738 2866  973   28   55    0
##
```

```
## $RATCAT_A
## x
##    1    2    3    4    5    6    7    8    9   10   11   12   13   14 <NA>
##  701  601  909 1017 1052 1060 1052 1995 1886 1738 1681 1328 1341 7854    0
##
## $MARSTAT_A
## x
##     1     2     3     4     5     6     7     8     9  <NA>
## 10118   611     5  2495  3253   320  5170  1422   821     0
##
## $BMICAT_A
## x
##    1    2    3    4    9 <NA>
##  364 7518 8108 7742  483    0
##
## $SMKCIGST_A
## x
##     1     2     3     4     5     9  <NA>
##  1626   609  6133 15171     1   675     0
##
## $DISAB3_A
## x
##     1     2  <NA>
##  2735 21480     0
##
## $PHSTAT_A
## x
##    1    2    3    4    5    7    9 <NA>
## 4789 8215 7389 2970  845    4    3    0
##
## $REGION
## x
##    1    2    3    4 <NA>
## 4380 5240 8827 5768    0
```

```
dat <- dat[vars]   # now dat has only your 11 variables
```

Codebook

Variable	What it measures	How it is stored in the file
PHQCAT_A	**Target.** Depressive symptom severity (PHQ-8), derived by NCHS from the eight PHQ-8 items	1 = none/minimal, 2 = mild, 3 = moderate, 4 = severe; 8 = not ascertained (missing)
AGEP_A	Age in years (top-coded)	Number; 18–84 = years, 85 = 85+; 97 = refused, 98 = not ascertained, 99 = don't know
SEX_A	Sex	1 = male, 2 = female; 7 = refused, 8 = not ascertained, 9 = don't know

Variable	What it measures	How it is stored in the file
EDUCP_A	Educational attainment	01 = grade 0–11, 02 = 12th no diploma, 03 = GED, 04 = HS graduate, 05 = some college, 06 = associate (occ/tech), 07 = associate (academic), 08 = bachelor's, 09 = master's, 10 = professional/doctoral; 97 = refused, 98 = not ascertained, 99 = don't know
RATCAT_A	Family income-to-poverty ratio (imputed)	01–14, from 01 = 0.00–0.49 up to 14 = 5.00 or greater; 98 = not ascertained (0 cases)
MARSTAT_A	Marital status	1 = married/spouse present, 2 = married/spouse absent, 3 = married/spouse presence unknown, 4 = widowed, 5 = divorced, 6 = separated, 7 = never married, 8 = living with a partner; 9 = unknown (missing)
BMICAT_A	BMI category	1 = underweight, 2 = healthy weight, 3 = overweight, 4 = obese; 9 = unknown (missing)
SMKCIGST_A	Cigarette smoking status	1 = current every-day, 2 = current some-day, 3 = former, 4 = never, 5 = smoker/status unknown (1 case); 9 = unknown if ever smoked (missing)
DISAB3_A	Disability status (Washington Group short set)	1 = yes, 2 = no; 9 = don't know (0 cases)
PHSTAT_A	Self-rated general health	1 = excellent, 2 = very good, 3 = good, 4 = fair, 5 = poor; 7 = refused, 8 = not ascertained, 9 =

Numbers That Are Really Categories

Convert categories stored as numbers into factors with readable labels, using `factor()` with levels and labels. Check the result with `table(x, useNA = "always")`. Any code left out of levels becomes `NA`.

The first table shows the codes before recoding, and the second shows the result.

```
table(dat$SEX_A, useNA = "always")
```

```
##
##     1     2     7     9  <NA>
## 11215 12975     2    23     0
```

```
dat$SEX_A <- factor(
  dat$SEX_A, levels = c(1, 2),
  labels = c("male", "female")
)

table(dat$SEX_A, useNA = "always")
```

```
##
##   male female   <NA>
##  11215  12975     25
```

```
table(dat$MARSTAT_A, useNA = "always")
```

```
##
##     1     2     3     4     5     6     7     8     9  <NA>
## 10118   611     5  2495  3253   320  5170  1422   821     0
```

```
dat$MARSTAT_A <- factor(
  dat$MARSTAT_A, levels = c(1, 2, 3, 4, 5, 6, 7, 8),
  labels = c("married/spouse present", "married/spouse absent",
             "married/spouse presence unknown", "widowed", "divorced",
             "separated", "never married", "living with a partner"))

table(dat$MARSTAT_A, useNA = "always")
```

```
##
##          married/spouse present           married/spouse absent
##                           10118                             611
## married/spouse presence unknown                         widowed
##                               5                            2495
##                        divorced                       separated
##                            3253                             320
##                   never married           living with a partner
##                            5170                            1422
##                            <NA>
##                             821
```

```
table(dat$BMICAT_A, useNA = "always")
```

```
##
##    1    2    3    4    9 <NA>
##  364 7518 8108 7742  483    0
```

```
dat$BMICAT_A <- factor(
  dat$BMICAT_A, levels = c(1, 2, 3, 4),
  labels = c("underweight", "healthy weight", "overweight", "obese"))

table(dat$BMICAT_A, useNA = "always")
```

```
##
##    underweight healthy weight     overweight          obese           <NA>
##            364           7518           8108           7742            483
```

```
table(dat$SMKCIGST_A, useNA = "always")
```

```
##
##     1     2     3     4     5     9  <NA>
##  1626   609  6133 15171     1   675     0
```

```
dat$SMKCIGST_A <- factor(
  dat$SMKCIGST_A, levels = c(1, 2, 3, 4),
  labels = c("current every-day", "current some-day", "former", "never"))

table(dat$SMKCIGST_A, useNA = "always")
```

```
##
## current every-day  current some-day            former             never
##              1626               609              6133             15171
##              <NA>
##               676
```

```
table(dat$DISAB3_A, useNA = "always")
```

```
##
##     1     2  <NA>
##  2735 21480     0
```

```
dat$DISAB3_A <- factor(
  dat$DISAB3_A, levels = c(1, 2),
  labels = c("yes", "no"))

table(dat$DISAB3_A, useNA = "always")
```

```
##
##   yes    no  <NA>
##  2735 21480     0
```

```
table(dat$REGION, useNA = "always")
```

```
##
##    1    2    3    4 <NA>
## 4380 5240 8827 5768    0
```

```
dat$REGION <- factor(
  dat$REGION, levels = c(1, 2, 3, 4),
  labels = c("Northeast", "Midwest", "South", "West"))

table(dat$REGION, useNA = "always")
```

```
##
## Northeast   Midwest     South      West      <NA>
##      4380      5240      8827      5768         0
```

Text That Should Be a Number

Convert numbers stored as text into numbers with as.numeric(). Count how many values became NA, and make sure only the entries you expected were lost.

I do not need to convert the only numeric variable, AGEP_A to numbers because all values are already numeric, even the code for missingness.

```
class(dat$AGEP_A)
```

```
## [1] "integer"
```

```
dat$AGEP_A[dat$AGEP_A %in% c(97, 98, 99)] <- NA
```

Recoding Ordinal Variables as Numeric

Decide whether ordinal variables, such as Likert items or education level, are treated as numbers or as factors, and explain your choice.

```
table(dat$EDUCP_A, useNA = "always")
```

```
##
```

```
##    1    2    3    4    5    6    7    8    9   10   97   99 <NA>
## 1428  377  572 5523 3391  955 2309 5738 2866  973   28   55    0
```

```
# 97 = refused, 98 = not ascertained, 99 = don't know -> NA; 01-10 kept numeric
dat$EDUCP_A[dat$EDUCP_A %in% c(97, 98, 99)] <- NA

table(dat$EDUCP_A, useNA = "always")
```

```
##
##    1    2    3    4    5    6    7    8    9   10 <NA>
## 1428  377  572 5523 3391  955 2309 5738 2866  973   83
```

```
table(dat$RATCAT_A, useNA = "always")
```

```
##
##    1    2    3    4    5    6    7    8    9   10   11   12   13   14 <NA>
##  701  601  909 1017 1052 1060 1052 1995 1886 1738 1681 1328 1341 7854    0
```

```
# 98 = not ascertained -> NA; 01-14 income bands kept numeric
dat$RATCAT_A[dat$RATCAT_A %in% c(98)] <- NA

table(dat$RATCAT_A, useNA = "always")
```

```
##
##    1    2    3    4    5    6    7    8    9   10   11   12   13   14 <NA>
##  701  601  909 1017 1052 1060 1052 1995 1886 1738 1681 1328 1341 7854    0
```

```
table(dat$PHSTAT_A, useNA = "always")
```

```
##
##    1    2    3    4    5    7    9 <NA>
## 4789 8215 7389 2970  845    4    3    0
```

```
# 7 = refused, 8 = not ascertained, 9 = don't know -> NA; 1-5 kept numeric
dat$PHSTAT_A[dat$PHSTAT_A %in% c(7, 8, 9)] <- NA

table(dat$PHSTAT_A, useNA = "always")
```

```
##
##    1    2    3    4    5 <NA>
## 4789 8215 7389 2970  845    7
```

Fixing the Dichotomous Target

Here I recode missingness and collapse to two classes.

```
# 8 = not ascertained -> NA
dat$PHQCAT_A[dat$PHQCAT_A %in% c(8)] <- NA

# collapse to two classes; NA target stays NA (dropped later in Part 4)
dat$dep_modplus <- factor(
  dplyr::case_when(
    dat$PHQCAT_A %in% c(3, 4) ~ "mod_plus",
    dat$PHQCAT_A %in% c(1, 2) ~ "none_mild",
    TRUE                      ~ NA_character_
  ),
  levels = c("none_mild", "mod_plus")
)
```

```r
table(dat$dep_modplus, useNA = "always")
```

```
##
## none_mild  mod_plus      <NA>
##     21956      1663       596
```

```r
dat$PHQCAT_A <- NULL # Drop the original PHQCAT_A to avoid data leakage
```

```r
str(dat)
```

```
## 'data.frame':    24215 obs. of  11 variables:
##  $ AGEP_A     : int  71 67 19 71 55 52 80 23 45 74 ...
##  $ SEX_A      : Factor w/ 2 levels "male","female": 2 1 1 1 2 2 2 2 2 2 ...
##  $ EDUCP_A    : int  1 3 2 4 8 4 1 8 8 4 ...
##  $ RATCAT_A   : int  6 12 1 14 13 8 14 14 14 7 ...
##  $ MARSTAT_A  : Factor w/ 8 levels "married/spouse present",..: 1 5 7 1 7 1 1 7 1 4 ...
##  $ BMICAT_A   : Factor w/ 4 levels "underweight",..: 3 2 3 3 4 NA 2 3 4 2 ...
##  $ SMKCIGST_A : Factor w/ 4 levels "current every-day",..: 3 3 4 4 4 4 1 4 4 1 ...
##  $ DISAB3_A   : Factor w/ 2 levels "yes","no": 1 2 2 2 2 2 2 2 2 2 ...
##  $ PHSTAT_A   : int  3 4 3 4 2 1 4 1 2 4 ...
##  $ REGION     : Factor w/ 4 levels "Northeast","Midwest",..: 3 3 3 3 3 3 3 3 3 3 ...
##  $ dep_modplus: Factor w/ 2 levels "none_mild","mod_plus": 2 1 1 1 1 1 1 1 1 1 ...
```

3. Handle Missing Data

Recode placeholder codes, such as 9, 99, -99, empty texted, or "not reported", to `NA`. Recode a code only in the columns where it means missing, since a value like 9 can be a real score in another column.

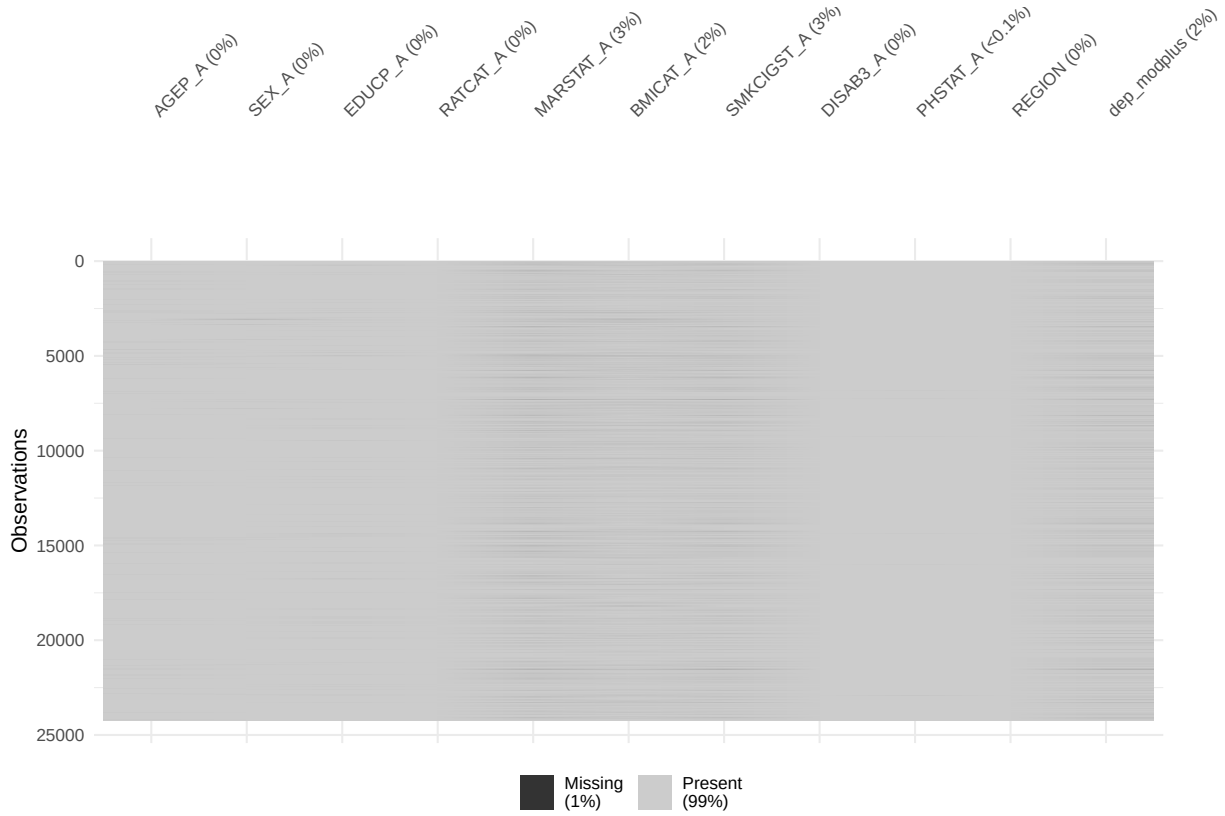
NHIS stores non-response as numeric codes (7/8/9, 97/98/99). These were converted to NA during recoding in Part 2, so no disguised missing values remain.

A First Picture

Visualize the missing data with `naniar::vis_miss()` and `naniar::gg_miss_var()`, and report the percentage missing for the target and for each predictor.

Black cells are `NA`.

```r
vis_miss(dat) + theme(plot.margin = margin(t = 40, r = 40, b = 10, l = 10))
```



For variables with substantial missingness, discuss whether the data are most lkely MCAR, MAR, or MNAR, and why.

Decide what to do with predictors that are missing for a large share of the sample. Drop them or keep them, and explain why.

Missing Values in Disguise

NHIS stores non-response as numeric codes (7/8/9, 97/98/99). These were converted to NA during recoding in Part 2, so no disguised missing values remain.

Drop rows where the target is missing (do not impute the target).

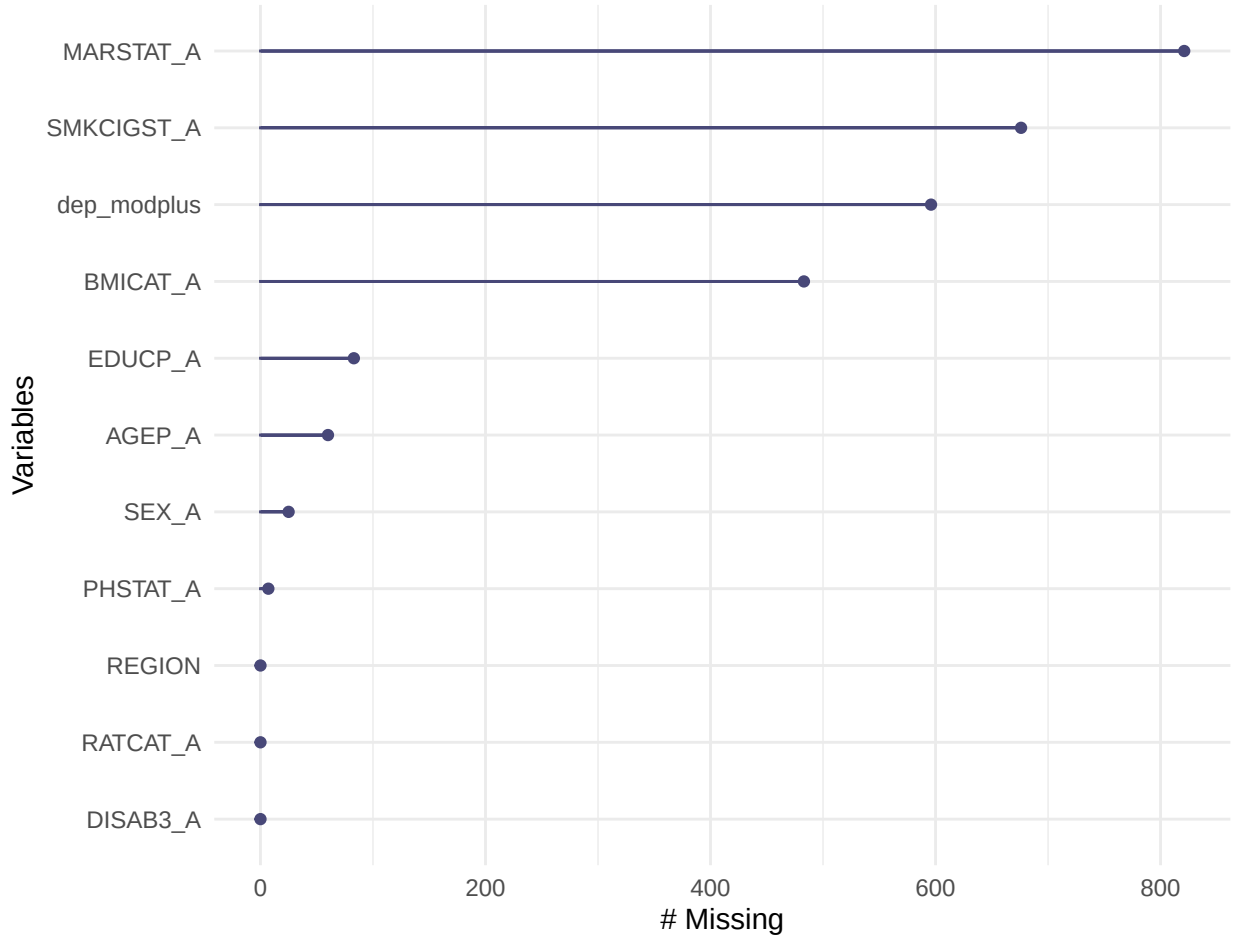
This was performed in part 3 when I handled missingness.

```
# Any value in any column that equals the code becomes NA
# dat <- replace_with_na_all(dat, condition = ~ .x == c(-99, -999))

# summary(dat$perceived_stress)
```

Where Are the Missing Values Now?

```
gg_miss_var(dat)
```



Imputation

Impute the remaining missing predictors inside your recipe, and justify the method you chose. (Options: `step_impute_median()` with `step_impute_mode()`; `step_impute_knn()`; `step_impute_bag()`.)

```
sum(is.na(dat$PHQCAT_A))
```

```
## [1] 0
```

Keep only the rows where `dep_modplus` is not missing. `is.na()` is `TRUE` for missing values, and `!` flips `TRUE` to `FALSE`.

```
nrow(dat)
```

```
## [1] 24215
```

```
dat <- dat[!is.na(dat$dep_modplus), ]

nrow(dat)
```

```
## [1] 23619
```

```r
round(colMeans(is.na(dat)) * 100, 2)   # % missing: target + each predictor
```

```
##      AGEP_A       SEX_A     EDUCP_A    RATCAT_A   MARSTAT_A    BMICAT_A
##        0.24        0.09        0.33        0.00        1.34        1.82
##  SMKCIGST_A    DISAB3_A    PHSTAT_A      REGION dep_modplus
##        0.70        0.00        0.03        0.00        0.00
```

Why is it a bad idea to impute the target? We do not impute the target because a participant with no outcome cannot teach the model anything, so those rows are dropped.

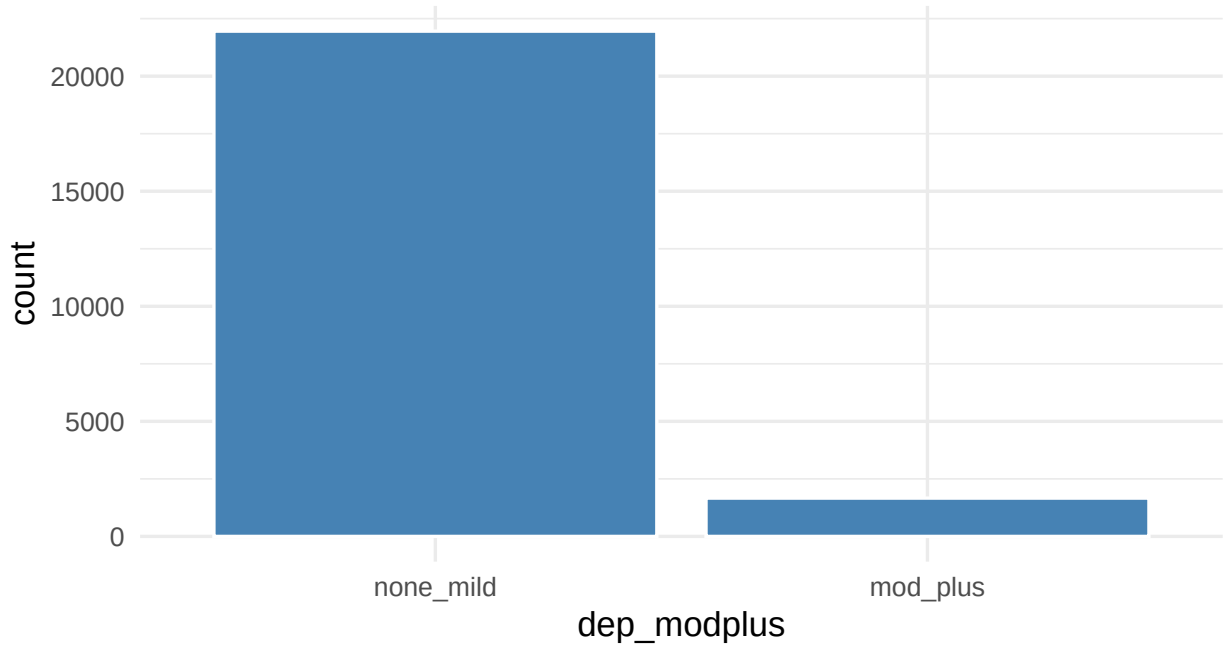
Now compare with listwise deletion, which would drop every student missing anything:

```r
sum(complete.cases(dat))
```

```
## [1] 22778
```

Listwise deletion would keep only 22778 of the 23619 responses Instead, the recipe in Part 6 fills in the missing predictors, so we keep all of them.

```r
ggplot(dat, aes(x = dep_modplus)) +
  geom_bar(fill = "steelblue", color = "white") +
  theme_minimal(base_size = 13)
```



4. Build the Modeling Pipeline

The recipe does four jobs, in this order:

1. **Fill missing numeric predictors with the median.**
2. **Fill missing categorical predictors with the most common category.**
3. **Dummy code the factors.**
4. **Drop any column with zero variance, then center and scale everything.**

Fill in the data argument and the selectors. The choices are `all_numeric_predictors()` and `all_nominal_predictors()`.

Split the data into training and test sets with `rsample::initial_split()`, stratified on the target. Set a seed so the split can be reproduced, and state the proportions you used.

```
set.seed(559)

split    <- initial_split(dat, prop = 0.5, strata = "dep_modplus")
train_df <- training(split)
test_df  <- testing(split)

c(train = nrow(train_df), test = nrow(test_df))
```

```
## train  test
## 11809 11810
```

Write a preprocessing recipe with the recipes package. At minimum, it should:

- **Impute missing predictors**
- **Dummy code factors**
- **Remove zero variance predictors**
- **Center and scale numeric predictors**
- **Other options: lump rare categories with `step_other()` or transforming skewed predictors**

```
# In recipes, "nominal" means factor (text columns count too), so
# all_nominal_predictors() selects the factors you made in Part 2
blueprint <- recipe(dep_modplus ~ ., data = train_df) %>%
  step_impute_median(all_numeric_predictors()) %>%
  step_impute_mode(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())

blueprint
```

Pass the recipe to `caret::train()` rather than preprocessing the data yourself, so every step is learned within each cross-validation fold.

`train()` will prep the recipe inside each fold for us. Here we prep and bake it once, only to check the result.

```
baked <- bake(prep(blueprint), new_data = NULL)
dim(baked)
```

```
## [1] 11809    23
```

```
sum(is.na(baked))
```

```
## [1] 0
```

There are no missing values left, and there are more columns than we started with, because dummy coding turned each factor into several indicator columns.

5. Fit and Compare Models

A Lambda Grid for This Data

```
x_baked <- as.matrix(baked[, names(baked) != "dep_modplus"])

lambda_max <- max(glmnet(x_baked, baked$dep_modplus,
```

```
                          family = "binomial", alpha = 1)$lambda)

lambda_grid <- 10^seq(log10(lambda_max) - 4, log10(lambda_max) + 1,
                      length.out = 100)

range(lambda_grid)
```

```
## [1] 6.182039e-06 6.182039e-01
```

Use k-fold cross validation with `trainControl()`, and state the number of folds.

Note any data leakage you are concerned about and what you did about it.

Fit a baseline model with `method = "null"`. It shows how well you can predict without using any predictors, and it is what we will compare every other model to.

Use half the responses for training and half for testing, as in the lecture.

`strata = "dep_modplus"` stratifies the split: it keeps the target's distribution about the same in the training and test sets. For a dichotomous target like this one, `initial_split()` groups the target into two sections. Both sets had the same split of none_mild to mod_plus.

Always stratify when your outcome is unbalanced, for example when only 10% of people are in one group. Otherwise the test set can end up with too few people in the smaller group to evaluate the model.

Everything in Parts 2 to 4 happened before the split, and that is fine. None of those steps learned anything from the data: `-99` means missing no matter who the student is. Imputation is different. It computes medians and modes from the data, so it belongs in the recipe, where it is learned from training data only.

The recipe takes the place of the formula `dep_modplus ~ .` in `train()`. It also does the centering and scaling, so we no longer need `preProc`.

```
ctrl <- trainControl(method = "cv", number = 10, classProbs = TRUE, summaryFunction = mnLogLoss) # retu
set.seed(559)
fit_null <- train(
  blueprint,
  data = train_df,
  method = "null", metric = "logLoss", trControl = ctrl)
```

One of the three must be the unpenalized regression, linear or logistic. At least one must be regularized: lasso, ridge, or elastic net, with a grid of lambda values (and alpha values for elastic net) tuned by cross validation. The others can use other regularization methods or different sets of predictors.

1. Ordinary logistic regression. *If your target is dichotomous, you are fitting logistic regression, and several settings change from the linear case. Before you fit your models, review the Loose Ends: Binary Outcome slide at the end of the Regularization lecture, which lists what needs to change.*

```
set.seed(559)
fit_glm <- train(
  blueprint,
  data = train_df,
  method = "glm", metric = "logLoss",
  trControl = ctrl)
```

Additional models: Lasso, Ridge, and Elastic Net

Compare at least three models in addition to the baseline. All of them must use the same target and the same training data.

A Quick Reference

All three models minimize the same thing:

$$\text{Loss} = \text{MSE}_{\text{train}} + \lambda \times \text{Penalty}$$

- `lambda` (λ) sets how much the penalty counts. Cross validation chooses it from the grid built in Part 6.
- `alpha` (α) sets which penalty is used.

Model	`alpha`	Penalty	What it does
Lasso	1	$\sum_{j=1}^{p} \lvert\beta_j\rvert$	Can shrink a coefficient to exactly zero, which drops that predictor
Ridge	0	$\sum_{j=1}^{p} \beta_j^2$	Shrinks every coefficient, but never to exactly zero
Elastic net	Between 0 and 1	$\alpha \sum_{j=1}^{p} \lvert\beta_j\rvert + \frac{1-\alpha}{2} \sum_{j=1}^{p} \beta_j^2$	A mix of the two, useful when predictors are highly correlated

Lasso

Set `alpha` for the lasso, and fill in the method and the grid.

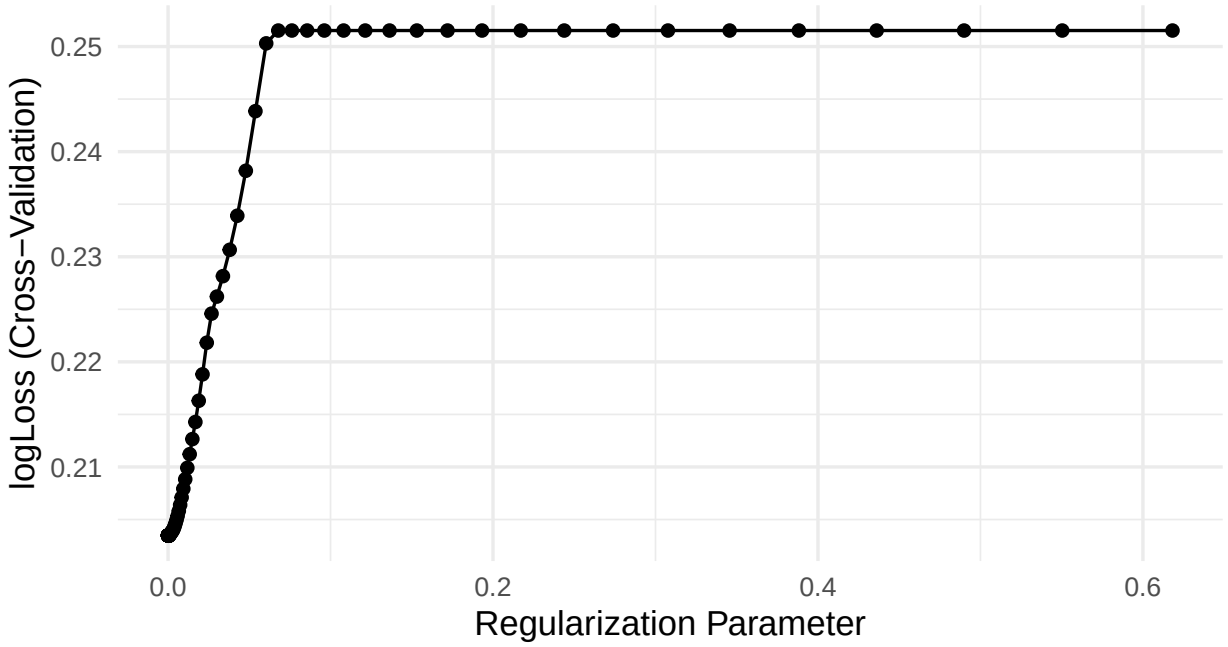
```
lasso_grid <- expand.grid(
  alpha  = 1,
  lambda = lambda_grid )

set.seed(559)
fit_lasso <- train(
  blueprint,
  data = train_df,
  method = "glmnet",
  family = "binomial", metric = "logLoss",
  trControl = ctrl,
  tuneGrid  = lasso_grid)

fit_lasso$bestTune
```

```
##    alpha       lambda
## 36     1 0.0003620833
```

```
ggplot(fit_lasso) + theme_minimal(base_size = 13)
```



Ridge

Set `alpha` for ridge.

```
ridge_grid <- expand.grid(
  alpha  = 0,
  lambda = lambda_grid )

set.seed(559)
fit_ridge <- train(
  blueprint,
  data = train_df,
  method = "glmnet",
  family = "binomial",
  metric = "logLoss",
  trControl = ctrl,
  tuneGrid  = ridge_grid )

fit_ridge$bestTune
```

```
##    alpha      lambda
## 60     0 0.005901056
```

Elastic Net

Let `alpha` run from 0 to 1 in steps of 0.1.

```
en_grid <- expand.grid(
  alpha  = seq(from = 0, to = 1, by = 0.1),
  lambda = lambda_grid)

set.seed(559)
fit_en <- train(
```

```
  blueprint,
  data = train_df,
  method = "glmnet",
  family = "binomial",
  metric = "logLoss",
  trControl = ctrl,
  tuneGrid  = en_grid)

fit_en$bestTune
```

```
##     alpha       lambda
## 443   0.4 0.0008172292
```

Comparing the Models

Every model was fit after `set.seed(559)`, so they all used the same 10 folds. This table puts their cross validated performance side by side. It is the table to use when choosing a final model, before the test set is touched. **Use performance measures that fit your target. For a continuous target, report RMSE, R2, and MAE. For a dichotomous target, report accuracy, sensitivity, and specificity.**

Include a table summarizing the cross validated performance of every model.

CV Accuracy Table

```
cv_table <- rbind(
  baseline    = getTrainPerf(fit_null),
  logistic    = getTrainPerf(fit_glm),
  lasso       = getTrainPerf(fit_lasso),
  ridge       = getTrainPerf(fit_ridge),
  elastic_net = getTrainPerf(fit_en)
)

round(cv_table[, "TrainlogLoss", drop = FALSE], 3)
```

```
##             TrainlogLoss
## baseline           0.252
## logistic           0.204
## lasso              0.203
## ridge              0.204
## elastic_net        0.203
```

```
cv_table[c("lasso","elastic_net"), "TrainlogLoss"]
```

```
## [1] 0.2034549 0.2034499
```

Choose a final model and justify the choice using the cross validation results. The final model I chose is the lasso model. I chose this model because although all models had relatively similar logLoss, the two best models (lasso and elastic net) were tied (both with a logLoss ~0.203)), and lasso is a much simpler model. Note: These differences are marginal, and all the models are pretty weak.

Evaluating the Final Model on The Test Set

Evaluate the final model on the test set, once. Report its test set performance. For a lasso or elastic net, report how many predictors it kept. You do not need to list the coefficients; the variable importance plot below shows which predictors matter.

Show which predictors matter most with `caret::varImp()`. The `vip` package used in lecture is no longer on CRAN.

In Project 1, choose your final model from the cross validation table in Part 8, and evaluate only that model on the test set.

```
fit_final <- fit_lasso   # chosen from cv_table: lowest TrainlogLoss and simplest model


pred_final <- predict(fit_final, newdata = test_df)

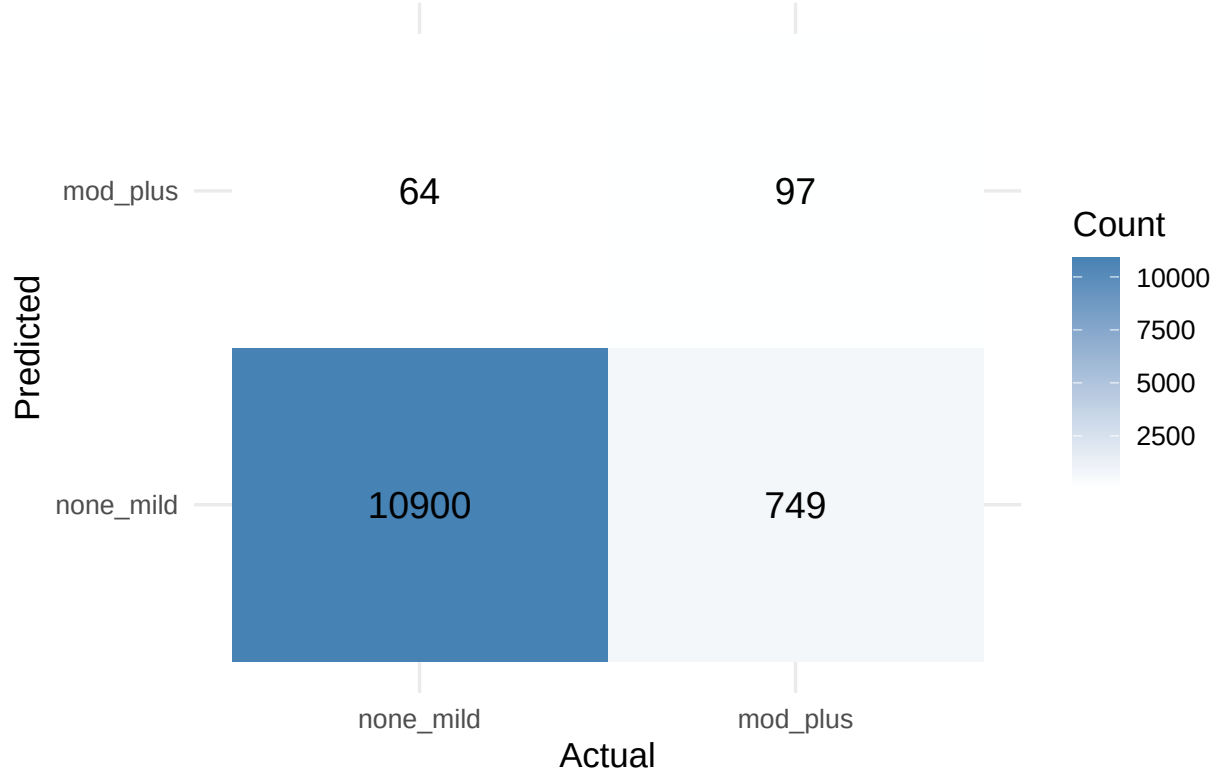
cm <- confusionMatrix(pred_final, test_df$dep_modplus, positive = "mod_plus")
cm
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction  none_mild mod_plus
##   none_mild     10900      749
##   mod_plus         64       97
##
##                Accuracy : 0.9312
##                  95% CI : (0.9264, 0.9357)
##     No Information Rate : 0.9284
##     P-Value [Acc > NIR] : 0.1227
##
##                   Kappa : 0.1737
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.114657
##             Specificity : 0.994163
##          Pos Pred Value : 0.602484
##          Neg Pred Value : 0.935703
##              Prevalence : 0.071634
##          Detection Rate : 0.008213
##    Detection Prevalence : 0.013633
##       Balanced Accuracy : 0.554410
##
##        'Positive' Class : mod_plus
##
```

```
library(ggplot2)

cm_df <- as.data.frame(cm$table)   # columns: Prediction, Reference, Freq

ggplot(cm_df, aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), size = 5) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  labs(x = "Actual", y = "Predicted", fill = "Count",
       title = "Confusion matrix: final lasso model (test set)") +
  theme_minimal(base_size = 13)
```



How many columns did the lasso keep, out of all the predictor columns it could use after dummy coding?

```
# columns the lasso kept
sum(coef(fit_lasso$finalModel, s = fit_lasso$bestTune$lambda)[-1] != 0)
```

```
## [1] 21
```

```
# columns it could use: everything in the baked data except symptoms
ncol(baked) - 1
```

```
## [1] 22
```

These are the coefficients the lasso kept. The recipe centered and scaled every predictor column, including the dummy columns, so the coefficients are on a standardized scale. Compare their sizes, but do not read them in raw units.

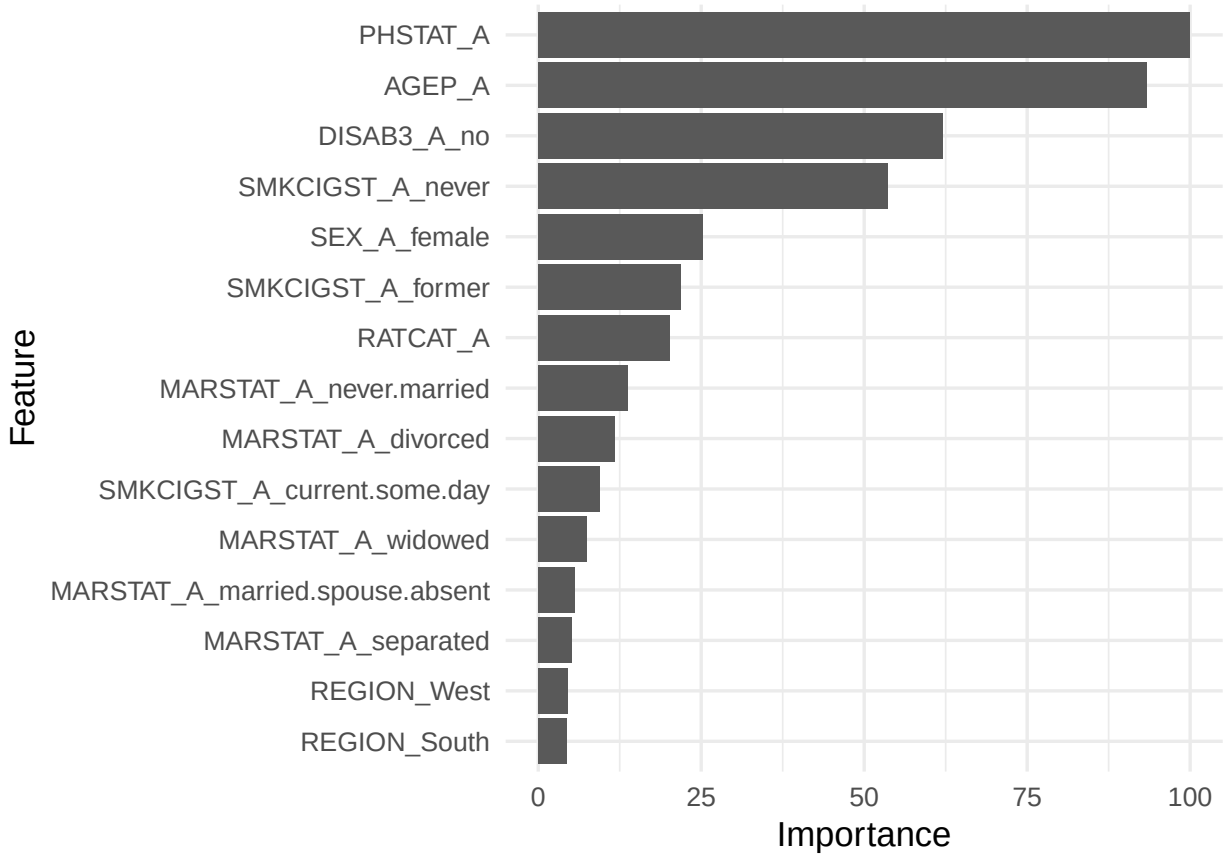
```
lasso_coefs <- as.matrix(coef(fit_lasso$finalModel, s = fit_lasso$bestTune$lambda))

round(lasso_coefs[lasso_coefs[, 1] != 0, , drop = FALSE], 3)
```

```
##                                          s=0.0003620833
## (Intercept)                                      -3.219
## AGEP_A                                           -0.694
## EDUCP_A                                           0.013
## RATCAT_A                                         -0.150
## PHSTAT_A                                          0.743
## SEX_A_female                                      0.188
## MARSTAT_A_married.spouse.absent                   0.042
```

```
## MARSTAT_A_married.spouse.presence.unknown          0.017
## MARSTAT_A_widowed                                  0.055
## MARSTAT_A_divorced                                 0.087
## MARSTAT_A_separated                                0.038
## MARSTAT_A_never.married                            0.102
## MARSTAT_A_living.with.a.partner                    0.028
## BMICAT_A_overweight                               -0.009
## BMICAT_A_obese                                     0.027
## SMKCIGST_A_current.some.day                       -0.070
## SMKCIGST_A_former                                 -0.163
## SMKCIGST_A_never                                  -0.399
## DISAB3_A_no                                       -0.461
## REGION_Midwest                                    -0.021
## REGION_South                                      -0.033
## REGION_West                                        0.034
```

```r
# varImp() comes with caret. It ranks predictors by the size of their
# standardized coefficients, scaled so the largest is 100, like vip() in lecture.
ggplot(varImp(fit_final), top = 15) +
  theme_minimal(base_size = 13)
```



Part 10: Wrapping Up

Answer each question in a sentence or two.

1. Which model had the highest performance (lowest CV log loss)? How many predictor columns did it keep?

The lasso had the lowest cross-validated log loss and was the final model; it kept 21 of 22 predictor columns.

2. Which predictors came out on top in the importance plot? Do they make sense?

   Self-rated health and age were the top predictors, followed by disability and never-smoker status; yes, these align with the depression literature.

3. Listwise deletion would have kept only 22778 of the 23619 responses Give one reason imputing inside the recipe is better than dropping those rows.

   Imputing helps us preserve the amount of data we have available for our work.